United International University
Department of Computer Science and Engineering

CSI 309/CSE 4509: Operating System Concepts/Operating Systems

Mid-term Examination: Fall 2025

Total Marks: 30 Time: 1 hour 30 minutes

Any examinee found adopting unfair means will be expelled from the trimester / program as per UIU disciplinary rules.

Answer all the questions. Numbers to the right of the questions denote their marks.

1. (a) A thread is usually defined as a light-weight process because OS maintains a smaller data structure for a thread than a process by sharing information among threads. Which one of the following is shared among threads? [2]

i. program counter	iv. heap
ii. code	v. global variable
iii. registers	vi. stack

(b) Draw the process state diagram for a scenario where processes are non-preemptive. Propose a solution if a process wants to move from a waiting/blocked state to a ready state, but there is no space in the ready queue. [2 + 1]

(c) Find the output of the following code: [3]

Process.c

```
int balance = 100;
int main() {
    int amount,cnt=0;
    do{
        wait(NULL);
        if(fork() || fork()){
            wait(NULL);
            amount = balance*0.1;
            balance += amount;
        }
        else{
            amount = balance*0.5;
            balance += amount;
            printf("Bye\n");
            exit(1);
        }
        if(cnt%2)break;
        cnt++;
    }while(fork());

    printf("Bal: %d\n",balance);
    return 0;
}
```

2. A computer laboratory uses a system to collect real-time data from a high-speed disk for processing large multimedia files. Initially, the system used interrupt-driven I/O, but the CPU became heavily loaded, causing delays in data processing and reduced system performance. To improve efficiency, the system administrator decided to implement Direct Memory Access (DMA) for data transfer between the disk controller and main memory. Based on the above scenario, answer the following questions within a few sentences.

(a) Why did the interrupt-driven I/O approach lead to high CPU utilization in this system? [1]

(b) Briefly explain how the introduction of DMA reduces CPU involvement during the data transfer. [1]

(c) Mention the role of the DMA controller in transferring data from the disk to main memory. Include the steps involved in the transfer process. [2]

(d) What happens when the DMA transfer is completed? How is the CPU notified? [1]

3. Consider the following four processes with the given arrival and burst time.

Process	Arrival Time (ms)	Burst Time (ms)
P1	0	22
P2	3	17
P3	7	25
P4	10	15

(a) Draw the Gantt chart using the STCF/SRTF scheduling algorithm. [4]

(b) Calculate the Turnaround Time (TAT) and Response Time (RT) for each process. [2]

(c) Describe the advantages and disadvantages of the STCF/SRTF algorithm. [2]

(d) (a) "Roud Robin algorithm optimizes turnaround time, while STCF/SRTF optimizes response time" - Do you agree or disagree? Explain your answer briefly. [2]

4. (a) What problem may occur if we use interrupt locking mechanism in a multi-core system? [2]

(b) Consider the following synchronization problem. A group of children are picking chocolates from a box that can hold up to N chocolates. A child that wants to eat a chocolate picks one from the box to eat, unless the box is empty. If a child finds the box to be empty, she wakes up the mother, and waits until the mother refills the box with N chocolates.
The synchronization for the child thread is done for you. You must now modify the code of the mother thread by adding suitable synchronization such that the mother invokes **refillChocolateBox(N)** only if the box is fully empty. You also need to give the initial values **(currently given as ? mark)** of the two semaphores. [1+4]

Synchronization.c

```
// Semaphores:
// chocolates : counting semaphore; number of chocolates currently available
// need_refill: semaphore used to wake the mother exactly when the box becomes
    empty
// mutex      : binary semaphore (mutex) to protect the shared variable count

semaphore chocolates = ?;     // initially the box is full
semaphore need_refill = ?;    // mother sleeps initially
semaphore mutex = 1;

int count = N;

// CHILD THREAD
while(True) {
    sem_wait(chocolates);        // blocks if box is empty (count==0)
    sem_wait(mutex);
    getChocolateFromBox();
    count--;

    if (count == 0) {
        sem_signal(need_refill);
    }
    sem_signal(mutex);
    eat();
}
```